

MIPS Architecture

A RISC-based Instruction Set Architecture



- **MIPS (Microprocessor without Interlocked Pipeline Stages)** architecture is a RISC (Reduced Instruction Set Computer) instruction set architecture (ISA) developed by MIPS Computer Systems.
- It is widely used in embedded systems, digital devices, and educational contexts for learning computer architecture concepts.

1. RISC Design Philosophy:

- MIPS follows the **RISC principles**, which focus on:
 - A small set of simple, uniform instructions.
 - Fixed instruction size (32 bits in standard MIPS).
 - Load/store architecture: Only load (LW) and store (SW) instructions access memory.
 - Simple addressing modes.

2. Instruction Formats: MIPS uses three main instruction formats:

- **R-type (Register-type)**: Used for arithmetic and logical instructions.
Example: $\text{ADD } \$d, \$s, \$t \rightarrow R[d] = R[s] + R[t]$
- **I-type (Immediate-type)**: Used for instructions with constants or addresses.
Example: $\text{ADDI } \$t, \$s, 10 \rightarrow R[t] = R[s] + 10$
- **J-type (Jump-type)**: Used for jump instructions.
Example: $\text{J } 10000 \rightarrow \text{Jump to address } 10000.$



3. Registers:

- MIPS has **32 general-purpose registers** (\$0 to \$31) of size 32 bits.
- Register \$0 always holds the value 0.
- Examples of registers:
 - \$s0-\$s7: Saved registers for preserving values.
 - \$t0-\$t9: Temporary registers for intermediate values.
 - \$a0-\$a3: Argument registers for passing parameters.
 - \$v0, \$v1: Return value registers.

4. Pipelining:

- MIPS uses **pipeline processing** to execute instructions efficiently.
- Instructions are divided into stages: **IF (Instruction Fetch)**, **ID (Instruction Decode)**, **EX (Execute)**, **MEM (Memory Access)**, and **WB (Write Back)**.

5. Load/Store Architecture:

- MIPS separates memory access operations (LW, SW) from computation operations (ADD, SUB, etc.).
- Only load and store instructions access memory; all computations occur in registers.

6. Endianness:

- MIPS supports both **big-endian** and **little-endian** byte ordering.

7. Instruction Set: The instruction set consists of:

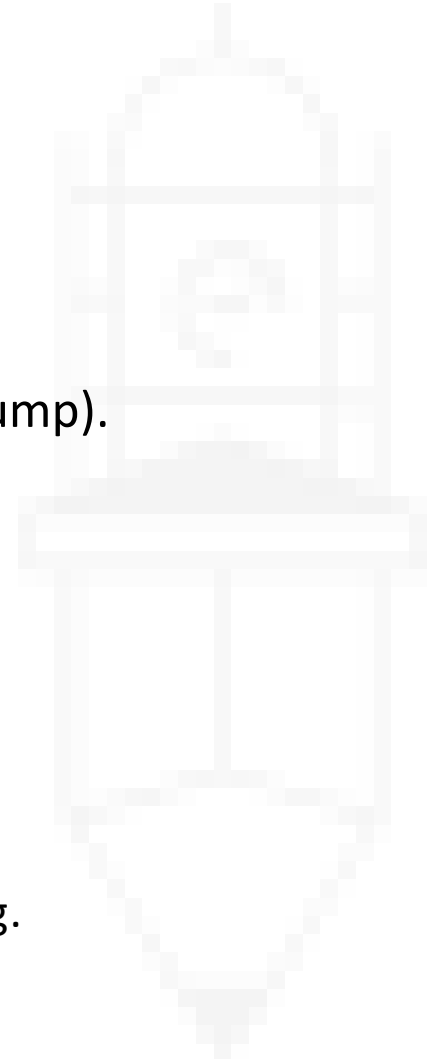
- **Arithmetic and logical instructions:** ADD, SUB, AND, OR, XOR.
- **Data transfer instructions:** LW (Load Word), SW (Store Word).
- **Control flow instructions:** BEQ (Branch if Equal), BNE (Branch if Not Equal), J (Jump).
- **Shift instructions:** SLL (Shift Left Logical), SRL (Shift Right Logical).

8. MIPS Variants:

- **MIPS32:** 32-bit version of MIPS.
- **MIPS64:** 64-bit version of MIPS.

9. Applications:

- Embedded systems, gaming consoles (e.g., PlayStation 2), routers, and academic learning.



- The **MIPS-I Memory Model** and **MMU (Memory Management Unit)** are critical components of the MIPS architecture, defining how memory is accessed and managed in MIPS-based systems.

1. MIPS-I Memory Model

I. Byte-Addressable Memory:

- The MIPS memory model is **byte-addressable**, which means that each memory address points to a single byte (8 bits).
- However, MIPS instructions typically work with **words** (4 bytes = 32 bits), so memory addresses must be aligned.

II. Aligned Memory Access:

- **Alignment** requires that word addresses (32-bit values) start at memory addresses that are multiples of 4.
- Misaligned accesses (e.g., reading a word from address (0x1001)) are not supported in MIPS-I and cause exceptions.
- **Example:**

Address:	0x1000	0x1001	0x1002	0x1003	← Word aligned
Value:	0xDE	0xAD	0xBE	0xEF	← 4 bytes (word)

III. Memory Segmentation:

- MIPS divides its address space into specific regions, separating **user mode** and **kernel mode** operations.
 - **User Address Space:** Typically ranges from 0x00000000 to 0x7FFFFFFF.
 - **Kernel Address Space:** Higher addresses, reserved for OS and privileged access.

Address Range	Description
0x00000000–0x7FFFFFFF	User space for applications.
0x80000000-0xFFFFFFFF	Kernel space (privileged mode).

IV. Endianness:

- MIPS supports both **Big-endian** and **Little-endian** byte orders.
 - **Big-endian:** Most significant byte (MSB) stored at the lowest address.
 - **Little-endian:** Least significant byte (LSB) stored at the lowest address.
- The system's endianness can be configured at startup.

2. MIPS-I MMU (Memory Management Unit)

- The **Memory Management Unit (MMU)** in MIPS is responsible for managing memory access, virtual memory, and address translation.

- **Key Features of the MMU:**

- I. **Virtual Memory Support:**

- The MMU allows processes to use **virtual addresses** instead of physical memory addresses.
 - Virtual memory provides an **illusion** of a large, contiguous memory space, even if physical memory is fragmented.

- II. **Page-Based Memory Management:**

- The memory is divided into **fixed-size pages** (commonly 4 KB).
 - The MMU maps virtual pages to physical frames in memory.
 - **Example:**

Virtual Address: 0x00401234 → Translates to Physical Address: 0x1F00234

III. Translation Lookaside Buffer (TLB):

- The MMU uses a **TLB** to speed up address translation.
- The TLB is a small cache that stores recently accessed page table entries, avoiding repeated lookups in the page table.

TLB Operations:

- **TLB Hit:** Address translation found in TLB → Fast access.
- **TLB Miss:** Address not in TLB → MMU performs a page table lookup.

IV. Page Faults and Exceptions:

- If a program accesses an invalid virtual address (e.g., unmapped page), the MMU raises a **page fault exception**.
- The OS handles the page fault, either by:
 - Loading the required page into physical memory.
 - Terminating the process if access is invalid.

V. Kernel vs. User Mode:

- The MMU separates **user mode** and **kernel mode** memory.
- **Kernel Mode:** Provides privileged access to manage the system (e.g., I/O, device drivers).
- **User Mode:** Restricts access to user processes to prevent unauthorized operations.

- In MIPS, **memory can only be accessed using load/store instructions**:

1. Load Instructions:

- Used to bring data from memory into registers.
- Example:
 LW \$t0, 4(\$s0) # Load a word from address (\$s0 + 4) into \$t0
 LB \$t1, 0(\$s1) # Load a byte from address (\$s1 + 0) into \$t1

2. Store Instructions:

- Used to write data from registers back to memory.
- Example:
 SW \$t2, 8(\$s0) # Store the word in \$t2 to address (\$s0 + 8)
 SB \$t3, 0(\$s1) # Store the byte in \$t3 to address (\$s1 + 0)

Summary

- The **MIPS-I memory model** uses a **byte-addressable**, word-aligned structure with support for both endianness modes.
- Memory is divided into **user space** and **kernel space**, providing security and privilege separation.
- The **MMU** in MIPS manages virtual memory, translating virtual addresses to physical addresses via page-based mapping and TLB.
- Load and store instructions provide controlled access to memory, adhering to the **load/store architecture** principle of MIPS.

- The **MIPS-I instruction set** is a part of the MIPS architecture and follows the **RISC (Reduced Instruction Set Computer)** design philosophy.
- It includes a simple and fixed set of instructions for efficient hardware implementation and software execution.
- The MIPS-I instruction set consists of **32-bit fixed-length instructions** divided into three main formats:
 1. **R-type (Register)**: Used for arithmetic and logical operations.
 - Example: ADD \$d, \$s, \$t
 2. **I-type (Immediate)**: Used for data transfer and branching.
 - Example: LW \$t, offset(\$s)
 3. **J-type (Jump)**: Used for jump instructions.
 - Example: J target
- Each format corresponds to a specific class of operations, and they simplify instruction decoding and execution.
- **Types of Instructions:**
 1. Arithmetic: ADD, SUB
 2. Logical: AND, OR, XOR
 3. Data Transfer: LW, SW
 4. Branching/Control: BEQ, BNE, J
 5. Shifting: SLL, SRL

1. R-Type (Register-Type) Instructions

- Used for **arithmetic, logical, and shift operations** that involve registers.
- Operates on data stored in registers.

Instruction Format:

Opcode (6)	rs (5)	rt (5)	rd (5)	shamt (5)	funct (6)
------------	--------	--------	--------	-----------	-----------

- **Opcode:** Specifies the operation type (always 0 for R-type instructions).
- **rs:** Source register 1.
- **rt:** Source register 2.
- **rd:** Destination register where the result is stored.
- **shamt:** Shift amount (used for shift instructions).
- **funct:** Function code to determine the exact operation (e.g., ADD, SUB).

Examples:

- **ADD:** ADD \$rd, \$rs, \$rt → Adds two registers.
ADD \$t0, \$t1, \$t2 # \$t0 = \$t1 + \$t2
- **SUB:** SUB \$rd, \$rs, \$rt → Subtracts two registers.
SUB \$s0, \$s1, \$s2 # \$s0 = \$s1 - \$s2

- **Logical Operations:**
 - AND: AND \$rd, \$rs, \$rt
 - OR: OR \$rd, \$rs, \$rt
- **Shift Operations:**
 - SLL: SLL \$rd, \$rt, shamt (Shift Left Logical).
SLL \$t0, \$t1, 2 # \$t0 = \$t1 << 2 (shift left by 2 bits)

2. I-Type (Immediate-Type) Instructions

- Used for **immediate values, memory access, and branching**.
- Includes a constant or address offset (immediate value).
- **Instruction Format:**

Opcode (6)	rs (5)	rt (5)	Immediate (16)
------------	--------	--------	----------------

- **Opcode:** Specifies the operation type.
- **rs:** Source register.
- **rt:** Destination register (or source/destination for memory instructions).
- **Immediate:** 16-bit constant or address offset.

Examples:

1. Immediate Arithmetic:

- **ADDI**: ADDI \$rt, \$rs, immediate → Adds immediate to a register.
ADDI \$t0, \$t1, 10 # \$t0 = \$t1 + 10

2. Memory Access Instructions (Load/Store):

- **LW** (Load Word): LW \$rt, offset(\$rs) → Loads a word from memory.
LW \$t0, 8(\$s0) # Load word at address \$s0 + 8 into \$t0
- **SW** (Store Word): SW \$rt, offset(\$rs) → Stores a word into memory.
SW \$t0, 8(\$s0) # Store word in \$t0 to address \$s0 + 8

3. Branch Instructions:

- **BEQ** (Branch if Equal): BEQ \$rs, \$rt, offset
BEQ \$t0, \$t1, LABEL # If \$t0 == \$t1, branch to LABEL
- **BNE** (Branch if Not Equal): BNE \$rs, \$rt, offset
BNE \$t0, \$t1, LABEL # If \$t0 != \$t1, branch to LABEL

3. J-Type (Jump-Type) Instructions

- Used for **unconditional jumps** to specific addresses.
- Contains a large target address for branching.
- **Instruction Format:**

Opcode (6)

Target Address (26)

- **Opcode:** Specifies the operation type.
- **Target Address:** 26-bit immediate address (combined with PC during execution).
- **Examples:**
 1. **Jump Instruction:**
 - **J:** J target → Jumps to the specified address.
J 10000 # Jump to address 10000
 2. **Jump and Link:**
 - **JAL:** JAL target → Jumps to address and saves return address in \$ra.
JAL FUNCTION # Call FUNCTION and save return address

MIPS-I includes the following types of instructions:

1. Arithmetic Instructions:

- ADD, ADDI, SUB, MUL, DIV
- Example:

ADD \$t0, \$t1, \$t2 # \$t0 = \$t1 + \$t2

2. Logical Instructions:

- AND, OR, XOR, NOR
- Example:

AND \$s0, \$s1, \$s2 # \$s0 = \$s1 AND \$s2

3. Shift Instructions:

- SLL (Shift Left Logical), SRL (Shift Right Logical), SRA (Shift Right Arithmetic)
- Example:

SLL \$t0, \$t1, 2 # \$t0 = \$t1 << 2

4. Data Transfer Instructions:

- LW (Load Word), SW (Store Word), LB (Load Byte), SB (Store Byte)
- Example:

LW \$t0, 4(\$s0) # Load word from memory SW \$t1, 8(\$s0) # Store word to memory

5. Branch and Jump Instructions:

- BEQ (Branch if Equal), BNE (Branch if Not Equal)
- J (Jump), JAL (Jump and Link), JR (Jump Register)
- Example:

BEQ \$t0, \$t1, TARGET # Branch to TARGET if \$t0 == \$t1

6. Special Instructions:

- **NOP** (No Operation): Executes no action.

Visual Representation of Formats

Format	Opcode	rs	rt	rd / Immediate / Target
R	6 bits	5 bits	5 bits	rd, shamt, funct (11 bits)
I	6 bits	5 bits	5 bits	16-bit immediate
J	6 bits	26-bit target address		

- The **MIPS Register Convention** defines how registers are used and what each register's role is within a program.
- MIPS architecture follows a **RISC (Reduced Instruction Set Computer)** design, which means it relies heavily on a small set of fast registers for computation.
- MIPS has **32 general-purpose registers**, each **32 bits wide** (in MIPS32), and they are denoted as **\$0 to \$31**.
- To ensure consistency in programming and facilitate function calls, these registers are assigned **specific conventions**.

- **Register Overview**
- MIPS registers are classified based on their **usage convention** into the following categories:

Register Name	Number	Usage
Zero Register	\$zero	Always contains the value 0.
Assembler Reserved	\$at	Reserved for assembler usage.
Function Arguments	\$a0–\$a3	Pass arguments to functions.
Temporary Registers	\$t0–\$t9	Temporary variables (caller-saved).
Saved Registers	\$s0–\$s7	Preserved across function calls.
Return Value Registers	\$v0–\$v1	Store return values from functions.
Global Pointer	\$gp	Points to global/static data.
Stack Pointer	\$sp	Points to the top of the stack.
Frame Pointer	\$fp	Points to the base of the stack frame.
Return Address	\$ra	Stores the return address for functions.

- **MIPS has 32 general-purpose registers:**
 - **\$0**: Always holds 0.
 - **\$at**: Reserved for assembler.
 - **\$v0-\$v1**: Function return values.
 - **\$a0-\$a3**: Arguments for function calls.
 - **\$t0-\$t9**: Temporary registers (caller-saved).
 - **\$s0-\$s7**: Saved registers (callee-saved).
 - **\$gp**: Global pointer.
 - **\$sp**: Stack pointer.
 - **\$fp**: Frame pointer.
 - **\$ra**: Return address.

- **Register Summary Table:**

Register	Name	Usage
\$0	Zero	Constant value 0
\$t0-\$t9	Temporaries	Temporary values
\$s0-\$s7	Saved	Preserved across calls
\$a0-\$a3	Arguments	Function arguments
\$v0-\$v1	Return values	Function return values

MIPS Register Breakdown

a) Special Purpose Registers

1. \$zero (Register 0):

- Always contains the value 0.
- Used when you need a constant zero value in operations.
- Example:

`ADD $t0, $t1, $zero # $t0 = $t1 + 0`

2. \$at (Assembler Temporary):

- Reserved for the assembler (pseudo-instructions).
- Not intended for general use by programmers.

b) Function Argument Registers

1. \$a0–\$a3 (Registers 4–7):

- Used to pass up to **4 arguments** to a function.
- If a function has more than 4 arguments, additional arguments are passed on the stack.
- Example:

`ADDI $a0, $zero, 10 # Pass 10 as the first argument`

c) Temporary Registers

1. \$t0–\$t9 (Registers 8–15, 24–25):

- Used for **temporary variables** within a function.
- **Caller-saved**: If a function needs these registers, it must save them before making a function call.
- Example:

ADD \$t0, \$t1, \$t2 # Temporary result in \$t0

d) Saved Registers

1. \$s0–\$s7 (Registers 16–23):

- Used for variables that need to be **preserved** across function calls.
- **Callee-saved**: If a function modifies these registers, it must restore them before returning.
- Example:

ADD \$s0, \$s1, \$s2 # \$s0 is preserved

e) Return Value Registers

1. \$v0–\$v1 (Registers 2–3):

- Used to **return values** from functions.
- If a function returns a single value, it is stored in \$v0.
- Example:

ADD \$v0, \$t0, \$t1 # Function result in \$v0

f) Stack Management Registers

1. \$sp (Stack Pointer):

- Points to the **top of the stack**.
- Used for function calls and local variables.
- Example:

SW \$t0, 0(\$sp) # Store \$t0 at top of the stack

2. \$fp (Frame Pointer):

- Points to the **base of the current stack frame**.
- Useful for accessing local variables within functions.

g) Return Address Register

1. \$ra (Return Address):

- Stores the **return address** when a function is called.
- The return address is the instruction immediately after the JAL (Jump and Link) instruction.
- Example:

JAL FUNCTION # Jump to FUNCTION and save return address in \$ra JR \$ra # Return to saved address

h) Global Pointer

1. \$gp (Global Pointer):

- Points to the middle of the **static data segment**.
- Used to access global or static variables efficiently.

Register Usage Rules

1. Caller-Saved Registers:

- \$t0–\$t9, \$a0–\$a3, \$v0–\$v1.
- The **caller** saves these registers before calling a function if needed.

2. Callee-Saved Registers:

- \$s0–\$s7.
- The **callee** (called function) saves and restores these registers.

3. Special-Purpose Registers:

- \$sp must always point to the top of the stack.
- \$ra must store the return address for proper function execution.

Register Convention Summary

Register	Number	Usage	Convention
\$zero	0	Constant zero	Always 0
\$at	1	Reserved for assembler	Assembler-specific
\$v0–\$v1	2–3	Function return values	Caller saves
\$a0–\$a3	4–7	Function arguments	Caller saves
\$t0–\$t9	8–15, 24–25	Temporary variables	Caller saves
\$s0–\$s7	16–23	Saved variables	Callee saves
\$gp	28	Global pointer	Static data access
\$sp	29	Stack pointer	Function stack
\$fp	30	Frame pointer	Base of stack frame
\$ra	31	Return address	Function return address

Example Program:

Adding two numbers using MIPS assembly.

.data

A: .word 5

B: .word 7

C: .space 4

First number

Second number

Memory for result

.text

main:

lw \$t0, A

lw \$t1, B

add \$t2, \$t0, \$t1

sw \$t2, C

li \$v0, 10

syscall

Load A into \$t0

Load B into \$t1

\$t2 = \$t0 + \$t1

Store result in C

Exit syscall